

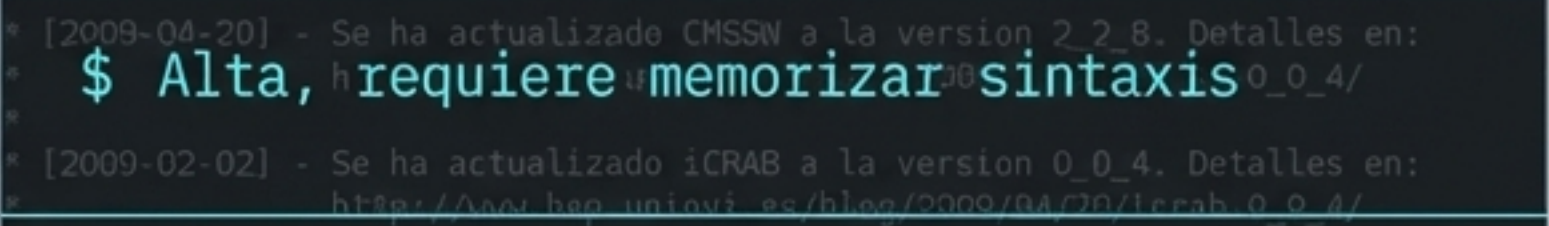
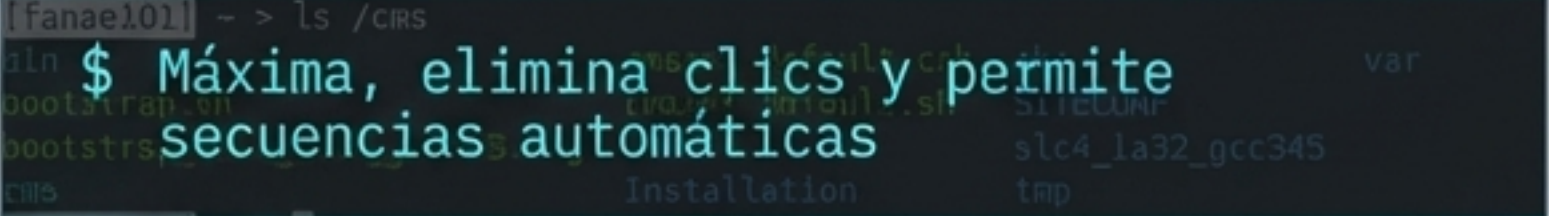
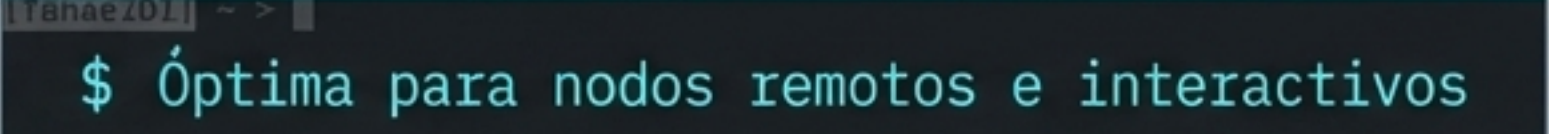


\$_ Arquitectura de Linux: De la Interfaz a la Consola

Guía visual para dominar el entorno, los permisos
y la lógica de la terminal científica.

Basado en el material de MSC. Juan Marcelo Gutierrez M.

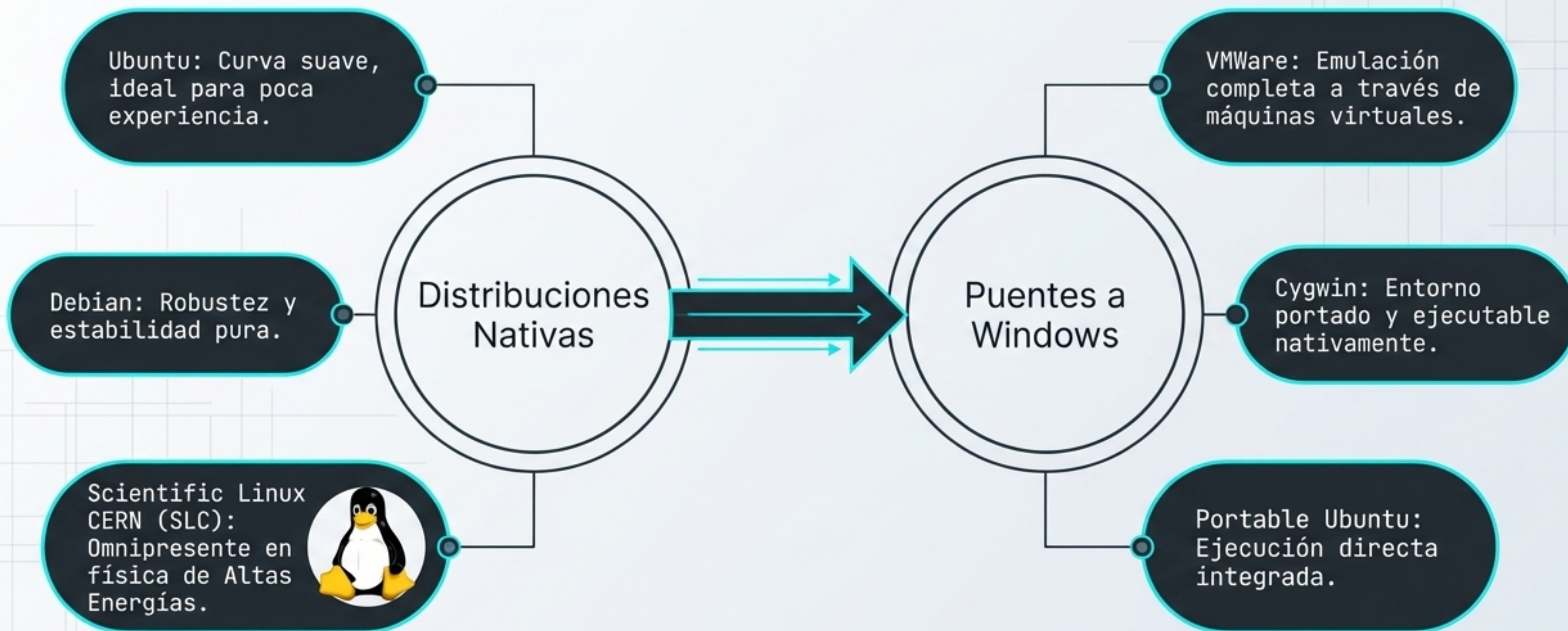
El Cambio de Paradigma: ¿Por qué abandonar el ratón?

Entorno Gráfico (GUI)	Consola (CLI)
Interfaz	
– Ventanas y clics, ej. Gnome/KDE	\$ Entrada de texto directa
Curva de Aprendizaje	
– Baja, intuitiva	\$ Alta, requiere memorizar sintaxis
Flexibilidad	
– Limitada por las opciones programadas visualmente	\$ Absoluta, sin restricciones de menús
Eficiencia y Automatización	
– Baja, requiere repetición manual	\$ Máxima, elimina clics y permite secuencias automáticas
Entornos Remotos	
– Lenta y pesada por red	\$ Óptima para nodos remotos e interactivos

Conclusión: En la ciencia y física de partículas, gran parte del trabajo es repetitivo. La **consola** no es un retroceso, es la **principal herramienta de optimización**.

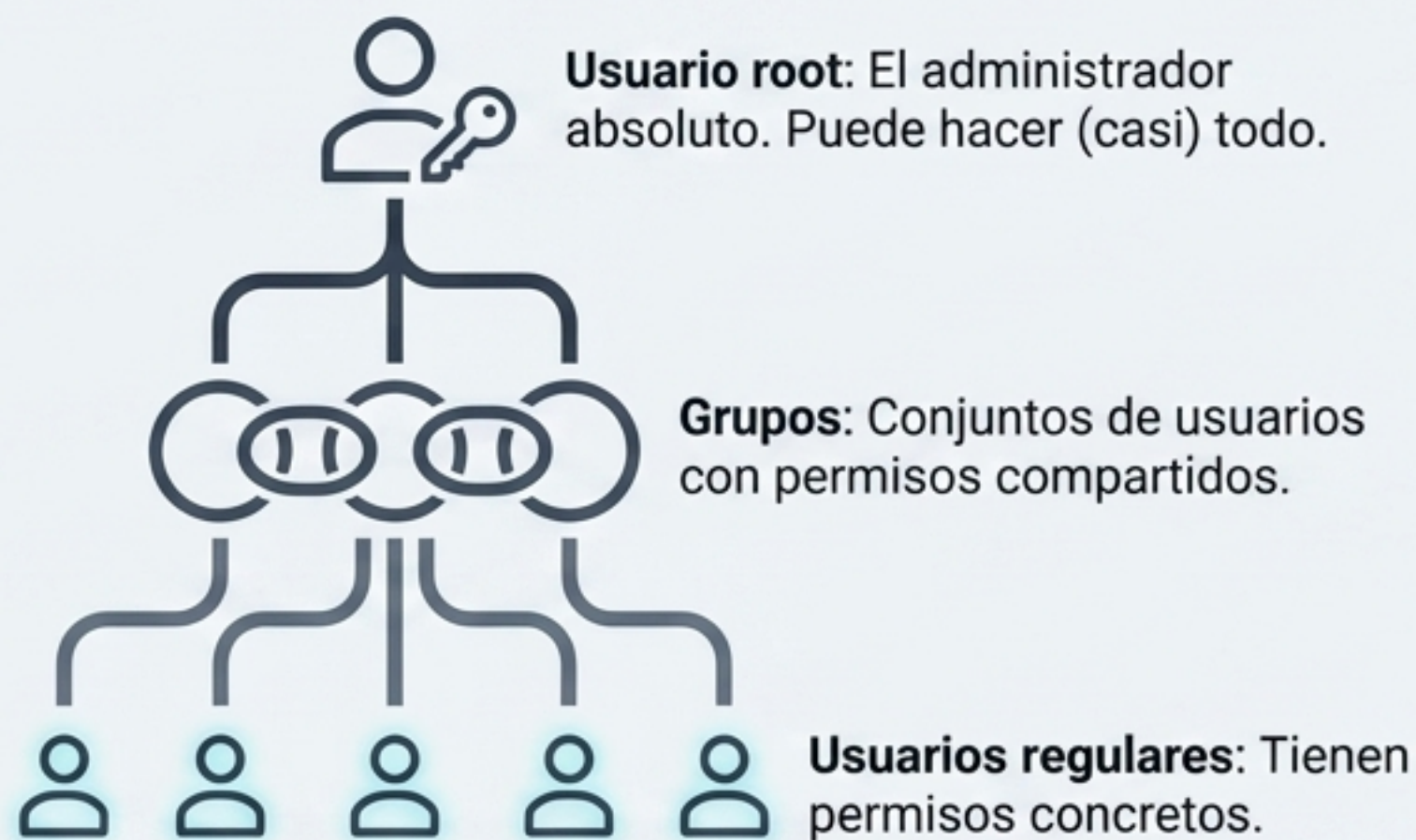
El Ecosistema: Distribuciones y Puentes

Concepto: GNU/Linux es un Kernel más herramientas. Las distribuciones empaquetan esto de distintas formas.



Fundamentos del Entorno: Usuarios y Directorios

La Jerarquía Multiusuario



Regla de Oro: Todos los ficheros pertenecen a algún usuario, quien controla su visibilidad.

Terminología Espacial

Ficheros = Archivos / Documentos

Directorios = Carpetas

\$ **.** → Directorio actual (Aquí)

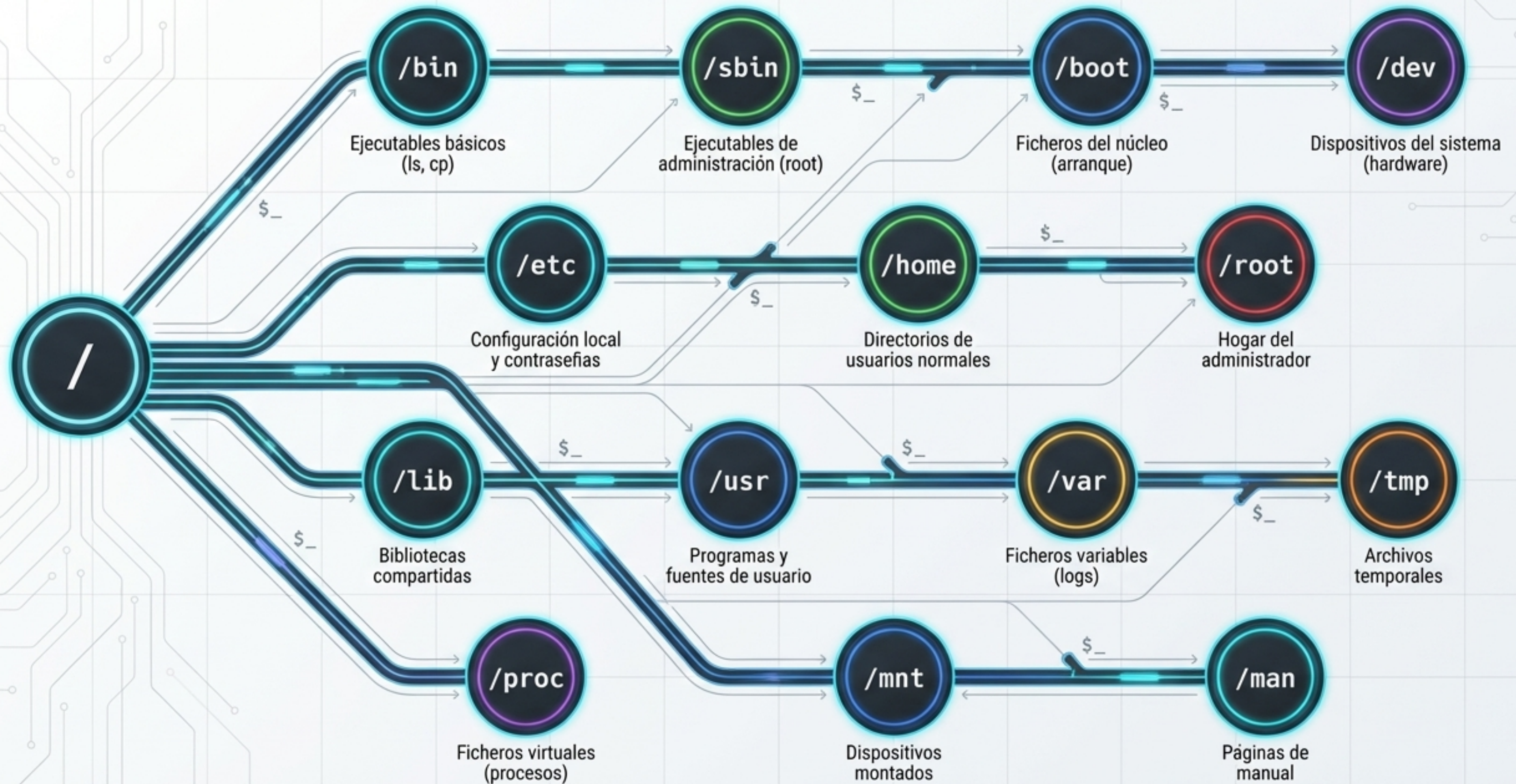
\$ **..** → Directorio superior (Arriba)

\$ **/** → Directorio raíz (Origen absoluto)

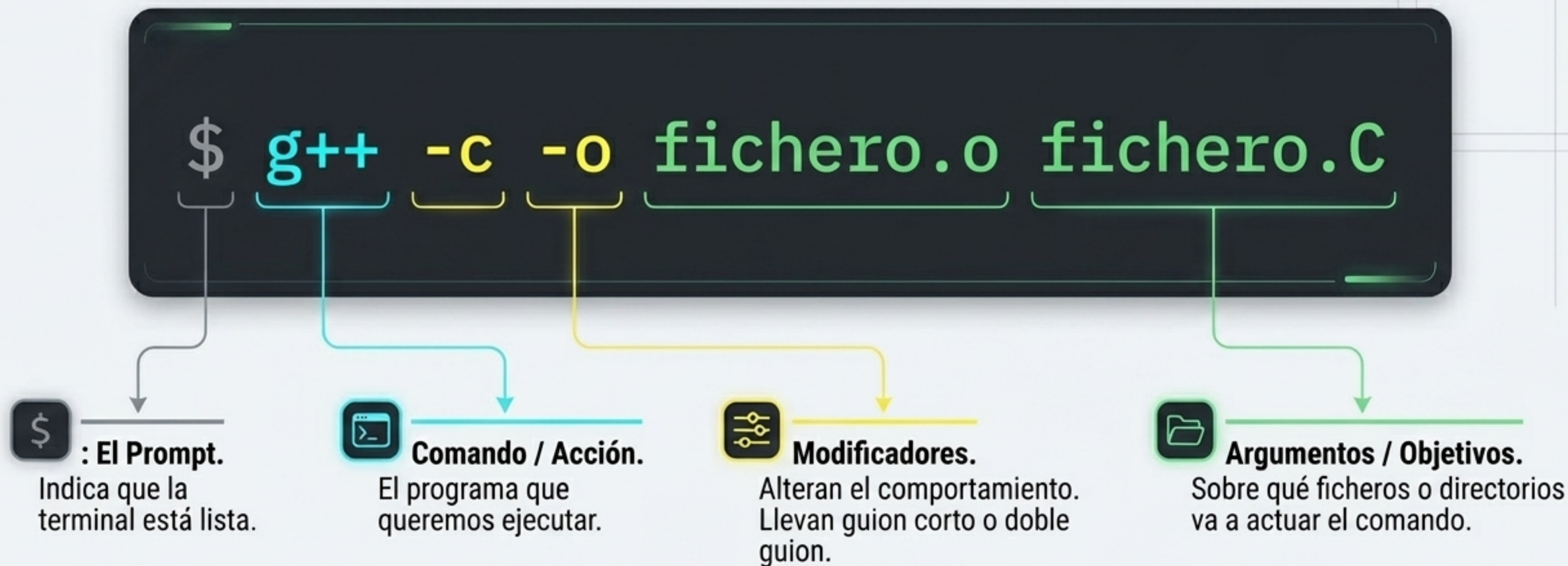
\$ **~** → Directorio de usuario (Hogar, ej: /home/larry)

—

Mapa Arquitectónico del Sistema de Archivos (/)



La Sintaxis de la Consola: Anatomía de un Comando



Regla Clave: Los comandos y parámetros siempre se separan por espacios. Para encadenar directorios se usa el path (ej: dirorigen/ficheroorig).

Matriz de Comandos Fundamentales

Navegación Espacial

pwd: Ruta del directorio actual.

cd [dir]: Cambia de directorio.

mkdir [dir]: Crea directorio nuevo.

rmdir [dir]: Borra un directorio vacío.

Manipulación de Ficheros

cp [orig] [dest]: Copia un fichero (-r para directorios).

mv [orig] [dest]:
Mueve o renombra un fichero.

rm [fichero]: Borra fichero permanentemente.

Inspección de Directorios

ls: Muestra el contenido.

Modificadores de **ls**:

-l: Formato largo (permisos, tamaño).

-a: Muestra todos (incluye ocultos).

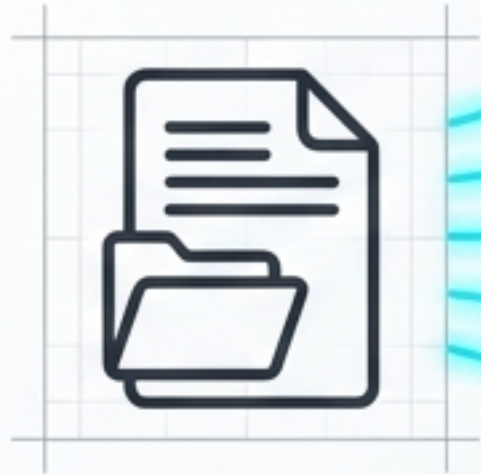
-t: Ordena por fecha.

-R: Búsqueda recursiva.

Nota: Las opciones se combinan (ej: **ls -alrt**).

Lectura de Archivos y Redirección

Herramientas de Visualización



`cat [fichero]`

Muestra todo el contenido de golpe (**-n** para numerar).

`more [fichero]`

Muestra por páginas (Avanza con Espacio).

`less [fichero]`

Visor avanzado (Navega con flechas, q para salir).

`wc [fichero]`

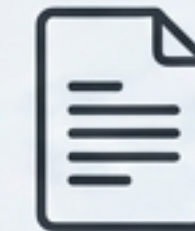
Cuenta líneas, palabras y caracteres.

`find [ruta] -name [patrón]`

Busca archivos específicos.

Redirección de Salida

`cat arch.txt`



nuevo.txt

Sobrescribe o crea un archivo nuevo con el resultado.

`cat arch.txt`



existe.txt

Añade (concatena) el resultado al final de un archivo existente.

El Decodificador de Permisos: Traducción de -rwxr-xr-x

-rwxr-xr-x

Tipo: - (Fichero),
d (Directorio),
l (Enlace)

Usuario (Dueño)

Grupo

Otros

👁 Lectura (r) = 4

✎ Escritura (w) = 2

⚡ Ejecución (x) = 1

Usuario (rwx)



Grupo (r-x)



Otros (r-x)



Matemática Octal (chmod)

Ejemplo Visual: `chmod 754`

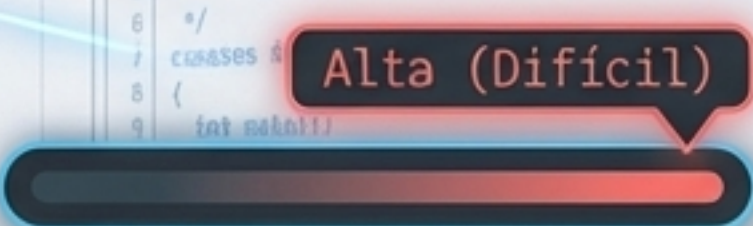
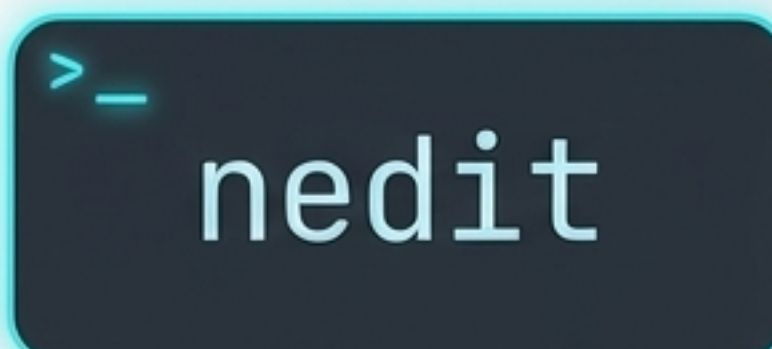
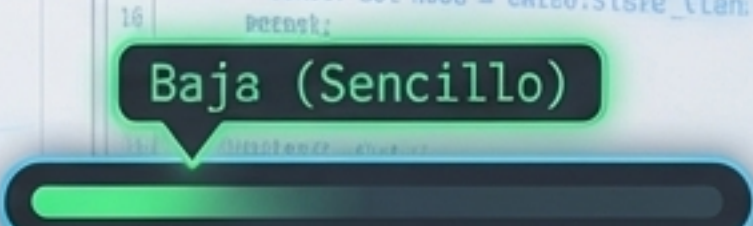
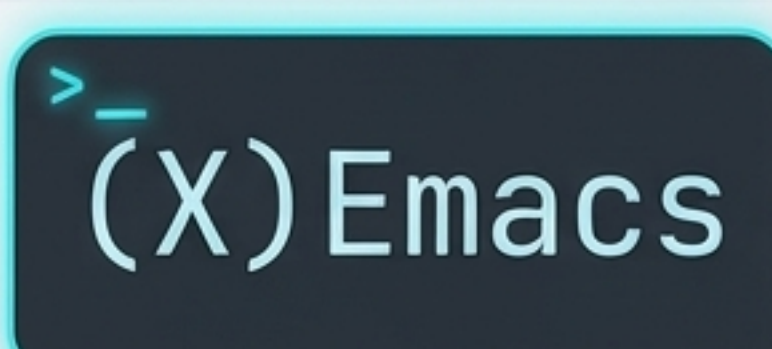
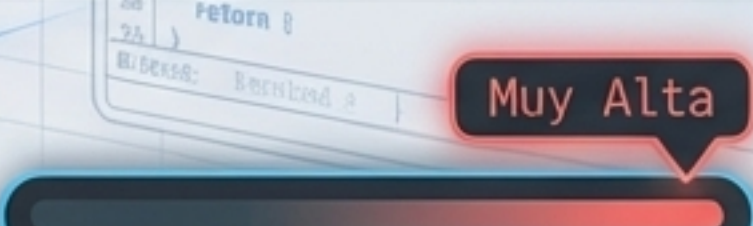
7 = 4+2+1 (rwx para Usuario)

5 = 4+1 (r-x para Grupo)

4 = 4 (r-- para Otros)

Uso alternativo: `chmod u+x fichero` (Añade ejecución al usuario).

El Arsenal de Edición: Eligiendo un Editor de Texto

Editor	Ubicación	Curva de Aprendizaje	Perfil
 vi	Universal (Instalado en todos los Linux)	 Alta (Difícil)	El estándar de supervivencia. Si el servidor falla, vi siempre estará ahí.
 nedit	Requiere instalación	 Baja (Sencillo)	Excelente balance entre funcionalidad y facilidad de uso inicial.
 (X)Emacs	Opcional / Instalable	 Muy Alta	Tremendamente potente y configurable. Un entorno de trabajo completo.

Dominio del Tiempo: Anatomía del Historial (history)



Cirugía de Argumentos

(Contexto: `$ cat arch1 arch2 arch3`)

`!!:1` → Llama solo al argumento 1 (arch1)

`!!:0-2` → Llama desde el comando (0) hasta el arg 2 (cat arch1 arch2)

`!!:2-` → Llama desde el arg 2 hasta el final (arch2 arch3)

Atajo de teclado vital: `ESC + .` inserta el último argumento en el prompt.

Comodines: El Motor de Búsqueda de Bash

Concepto Clave: Las interpretaciones las hace bash, no el comando. El comando solo ve los resultados filtrados.

?

Representa **exactamente un carácter cualquiera**.
(Ej: `/s??` encuentra `/srv`, `/sys`)

*

Representa **cero o más caracteres cualesquiera**.
(Ej: `/s*` encuentra `/sbin`, `/srv`, `/sys`)

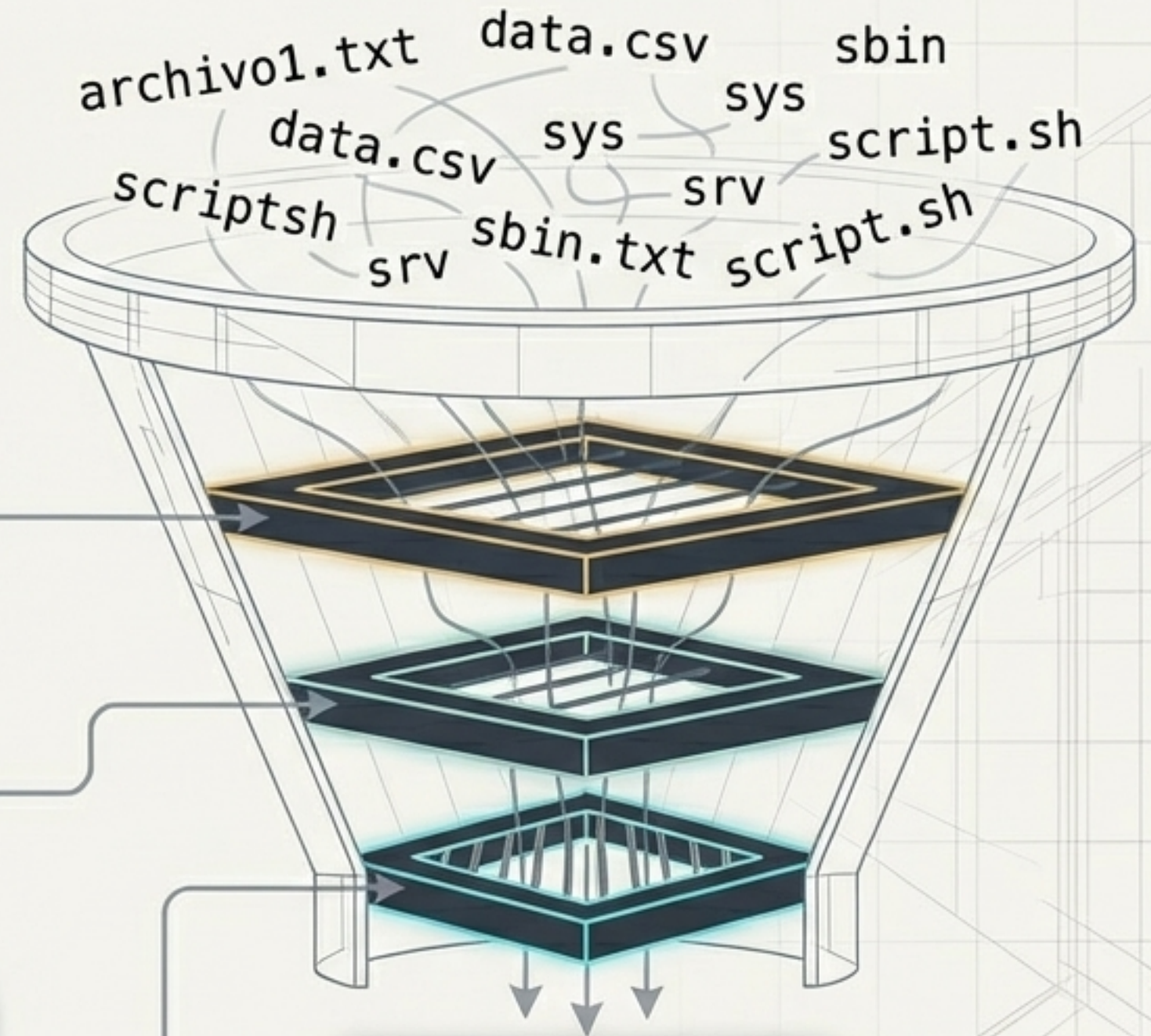
[]

Representa un **carácter dentro de un conjunto específico**.

`$_[by]` → Contiene 'b' o 'y' (Ej: `/s[by]*` → `/sbin`, `/sys`)

`$_[^by]` → Excluye 'b' e 'y' (Ej: `/s[^by]*` → `/srv`)

`$_[b-e]` → Rango válido: b, c, d, e

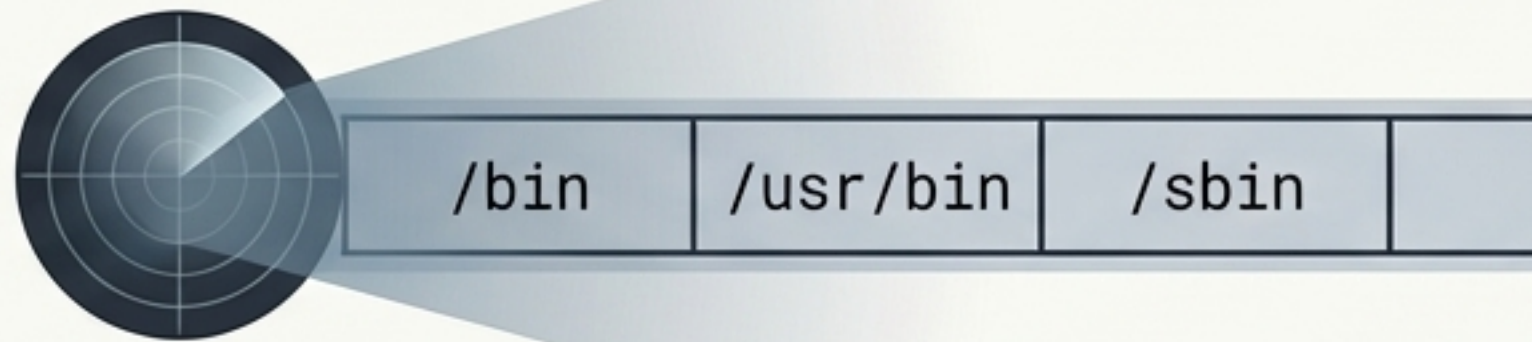


`/sbin`
`/srv`
`/sys`

Resultados pasados al comando (ej. `ls`)

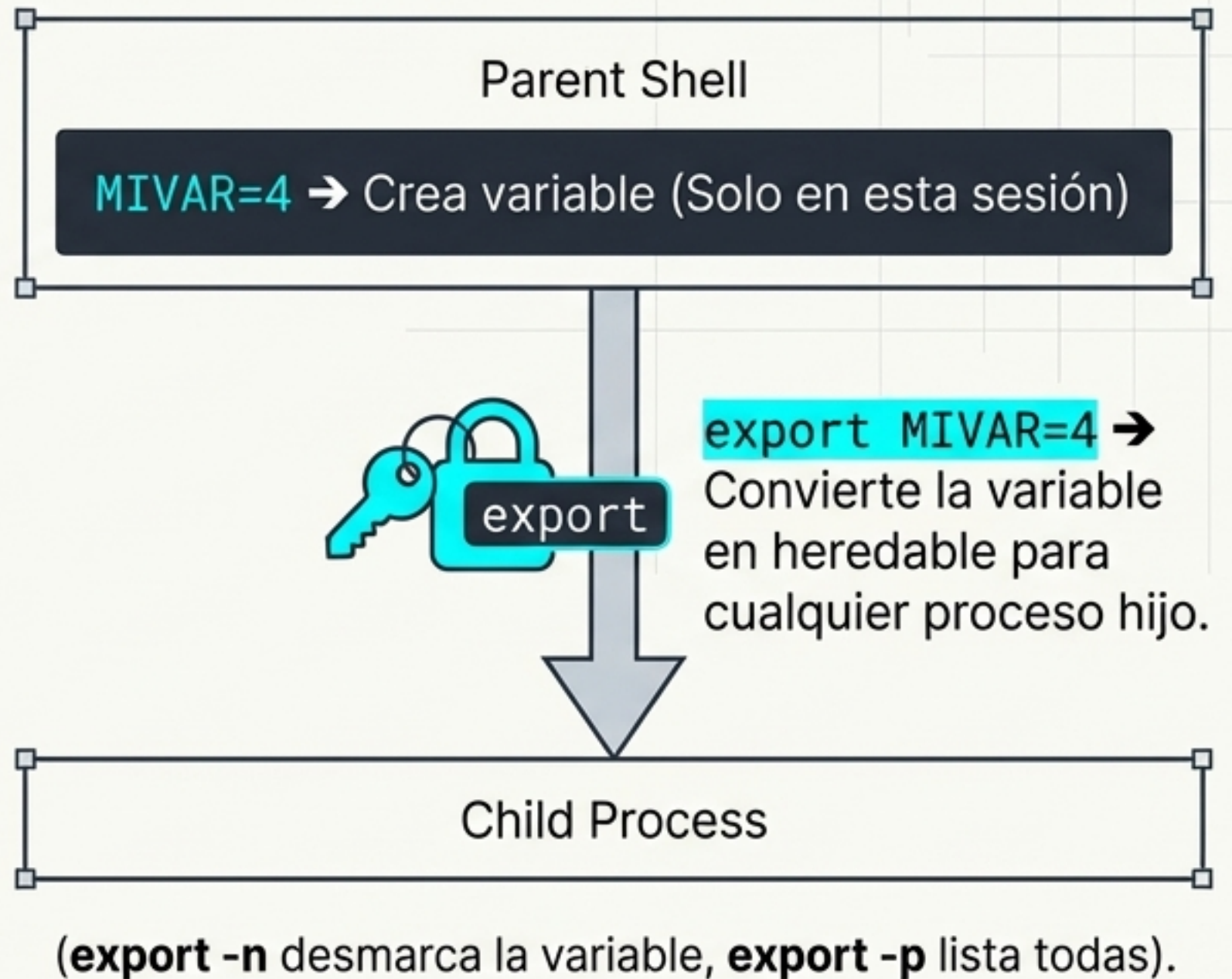
Variables de Entorno: El Cerebro del Sistema

El Poder de \$PATH



- \$ Es una lista de directorios separados por :
- \$ Al escribir `cp`, Bash busca el ejecutable de izquierda a derecha en `$PATH`.
- \$ Si no está en la lista, da error. Los scripts locales requieren `./mi_script.sh` porque el directorio actual `(.)` no suele estar en `$PATH`.

Creación y Herencia



Diagramas de Flujo en Consola: Operadores Lógicos

Separadores:
; ejecuta secuencias sin importar errores.
\ escapa el salto de línea para comandos largos.



\$ Ejecuta Comando 2 SÓLO SÍ
Comando 1 tuvo éxito.

```
true && echo 'Éxito' ✓
```

\$ Ejecuta Comando 2 SÓLO SÍ
Comando 1 fracasó.

```
false || echo 'Fracaso' ✗
```

Prioridad y Agrupación

```
{ Comando1 && Comando2; }
```

\$ Evaluación estricta de izquierda a derecha.
_ Para alterar el orden en la misma shell, agrupe
con llaves: { Comando1 && Comando2; }.

Anatomía de la Automatización: La Síntesis

El Entorno: Navegación dinámica independiente del usuario (Hogar).

Lógica: Condicional. Solo avanza si la carpeta data realmente existe.

Comodines: Filtro inteligente. Captura archivos que empiecen con 'data_', sigan con un número, y terminen en '.txt'.

```
$ cd ~/data && ls data_[0-9]*.txt > resumen.log || echo 'Error'
```

Redirección: Crea o sobrescribe resumen.log con la lista generada, sin ensuciar la pantalla.

Manejo de Errores: Si el directorio no existe, intercepta el fallo e imprime una advertencia segura.

Linux no es un conjunto de herramientas aisladas; es un **lenguaje modular** donde la salida de un proceso alimenta la **lógica del siguiente**.